RAG: answers from documents

Find the right paragraphs first,
then let a model write the answer

The chat on the shop answers policy questions with a keyword placeholder: it shows the paragraph that shares the most words with the question. In this step you replace it with RAG in `api/rag.py`. The first half is your Step 1 search, reused. The second half is new: a text model that writes an answer from what was found, and says "I don't know" when nothing was.

contents

1 What RAG is **2**
1.1 Two ways a model can answer . . . 2
1.2 Retrieval, then generation . . . 2
1.3 Semantic search and RAG side by side . . . 3

2 The documents and their chunks **3**
2.1 A policy document . . . 3
2.2 Chunks . . . 3

3 Retrieval with rules **4**
3.1 Meaning is not enough . . . 4
3.2 The topic: a preference, not a filter . . . 4
3.3 `retrieve` . . . 4

4 The prompt **5**

5 The instruction: being honest **5**
5.1 The answer as a form . . . 6

6 The exercise **6**

7 Experiments, after it passes **7**

1 What RAG is

1.1 *Two ways a model can answer*

Ask a text model "How many days do I have to return a sofa?" and it will answer. But from what? It has read a lot of the internet, so it will give a typical number, maybe 30, maybe 14. It does not know Nordhjem's rules, and it does not say that it does not know. It *sounds* sure either way. Inventing a fact in a confident voice is called a *hallucination*.

hallucination: a fluent, confident answer that is not true, because the model made it up.

The fix is to hand the model the rules with the question, and tell it to answer only from them. Like an open-book exam: the student still writes the answer, but from the book on the table, not from memory.

1.2 *Retrieval, then generation*

The shop has 23 policy documents, about 2 500 words. Sending all of them with every question would work today, but not with 2 000 documents, and the more text a model gets, the easier it misses the one line that matters. So the right paragraphs are *found* first.

That is *RAG*, in three words:

RAG: retrieval-augmented generation: find relevant text, add it to the prompt, then generate the answer.

- **Retrieval**: find the few paragraphs closest to the question. This is your Step 1 semantic search, on paragraphs instead of products.
- **Augmented**: put those paragraphs into the prompt, with the question.
- **Generation**: the text model writes the answer from them, and names its sources.

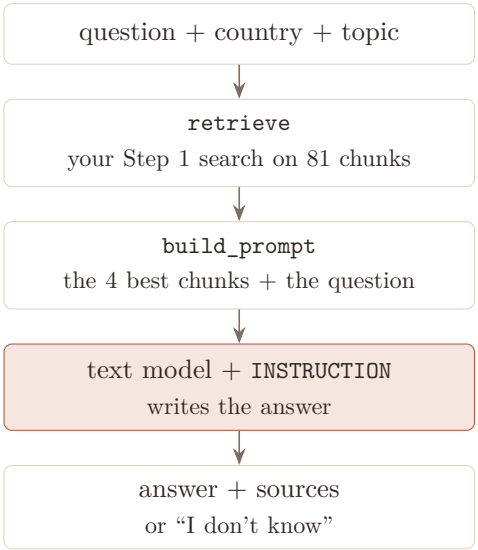

Figure 1 RAG in the shop. Only the highlighted step is new in kind.

1.3 *Semantic search and RAG side by side*

	Semantic search (Step 1)	RAG (Step 2)
Searches over	500 products	81 paragraphs of policy
Returns	a list of products	a written answer and its sources
Who reads the results	the customer	the text model, then the customer
Worst failure	a wrong product in the list	a wrong answer that sounds right

The last row is why this step has more rules. A wrong product in a list is easy to skip. A wrong answer in a confident sentence is believed.

2 The documents and their chunks

2.1 *A policy document*

The documents are in `data/policies/`. Each starts with a header, then Markdown text with `##` headings:

```
---
doc_id: delivery-no
title: Delivery in Norway
topic: delivery
country: NO
valid_from: 2026-01-01
---
# Delivery in Norway
## Price and time
- Standard delivery: **799 NOK**. There is no free delivery ...
## Returns from Norway
...
```

2.2 *Chunks*

A whole document is too coarse to retrieve: "Delivery in Norway" covers price, customs and returns. A single sentence is too fine: it loses its context. Here a *chunk* is one heading with its paragraphs. The 23 documents give 81 chunks.

chunk: the unit that is embedded and retrieved: one piece of a document.

`loaders/load_policies.py` has already embedded every chunk (as `RETRIEVAL_DOCUMENT`, normalised, like the products) and stored it in Firestore, in the collection `policy_chunks`. The embedded text includes the title and the heading, because a heading like "Price and time" means little without "Delivery in Norway":

```
Delivery in Norway. Price and time.
```

```
- Standard delivery: **799 NOK**. There is no free delivery ...
```

`load_chunks` in `api/rag.py` reads them into a `ChunkIndex`, the same idea as your `Index`: row i of every list is chunk i. It is given, since you wrote one already.

3 Retrieval with rules

3.1 Meaning is not enough

Some chunks must never reach the model, however close they are in meaning:

- **Another country.** A customer in Sweden asks "How much is delivery?". The Danish chunk (499 kr) is as close in meaning as the Swedish one (599 SEK). Only the Swedish one may be used.
- **An old version.** `returns-2025` (30 days) was replaced by `returns-2026` (60 days). Both answer "how long can I return?" equally well. Only the current one is true.

These are filters, exactly like `filter_products`: a yes or no per chunk, applied before ranking. That is TODO 1, `allowed_chunks`.

```
                countries  valid_to     DK on 2026-11-13
returns-2026    [all]      None         True
delivery-dk     [DK]       None         True
delivery-se     [SE]       None         False   other country
returns-2025    [all]      2025-12-31   False   expired
```

> Dates written year-month-day compare correctly as plain text: `"2025-12-31" < "2026-11-13"` is `True`. No date objects needed.

3.2 The topic: a preference, not a filter

The chat lets the customer pick a topic, such as *Returns*. It would be natural to filter on it. But the test question "How much is a return pickup in Norway?" is a returns question whose answer is in *Delivery in Norway*. A topic filter would hide it.

So the topic only adds a small bonus to the score of chunks in that topic: `TOPIC_BONUS = 0.03`. A chunk in the right topic moves up; a much better chunk elsewhere still wins.

```
scores = scores + TOPIC_BONUS * (index.topics == topic)
```

`index.topics == topic` gives True or False per chunk; times a number, True counts as 1 and False as 0.

3.3 `retrieve`

TODO 2 is your Step 1 `search` again: embed, normalise, score with `@`, add the topic bonus, `np.where` with $-\infty$ for the chunks that are not allowed, cap k,

`get_top_k`. The functions from `api/search.py` are imported, so you reuse your own code. `K = 4`: the four best chunks go to the model.

4 The prompt

The model gets two texts. The *instruction* (TODO 4) is the same for every question. The *prompt* (TODO 3) changes each time:

```
Customer's country: Norway (NO)

Sources:
[delivery-no] Delivery in Norway, Returns from Norway
You have the same 60 days to return an item. A return pickup in
Norway costs **599 NOK**. ...

[returns-2026] Returns policy, Cost of returning
Returning an item in a store is free. ...

Question: How much is a return pickup in Norway?
```

Three things in it do real work:

- **The country.** The chunks were already filtered by country, but a chunk for "all" can still mention several countries (the warranty chunk has both 2 and 5 years). The model needs to know which applies.
- **The `[doc_id]` labels.** The model answers with the ids it used. That is how the chat shows "Source: Delivery in Norway".
- **Several chunks.** Some answers need two: "Can I return a mattress after 80 nights?" needs the trial, which overrides the normal returns rule.

5 The instruction: being honest

The instruction decides whether the model stays inside the sources. The rules that matter most:

Rule	Without it
answer only from the sources	"Do you offer interior design?" gets a friendly, invented yes
if they do not answer, say "I don't know"	the model fills the gap from what shops usually do
list the doc_ids used	no way to show or check the source
use the customer's country	the Danish 2-year rule for a Norwegian customer
answer in the question's language	a Danish question gets English

"I don't know" is a right answer. Three test questions have no answer in the documents, and they pass only if the chat says so. Retrieval always returns four chunks, even for a question about renting furniture: only the instruction stops the model from building an answer out of whatever came back.

5.1 The answer as a form

As in Step 1b, the model fills in a form instead of writing free text:

```
class RagAnswer(BaseModel):
    answer: str
    sources: list[str]     # doc_ids, for example ["delivery-no"]
```

The model can still write an id that was not in the prompt. So `answer` keeps only ids that were among the retrieved chunks, and turns each into a `Source` with its title.

6 The exercise

Everything is in `api/rag.py`. Work from top to bottom:

1. **TODO 1**, `allowed_chunks`: country and dates, one True or False per chunk (section 3.1).
2. **TODO 2**, `retrieve`: your Step 1 search with the topic bonus (sections 3.2 and 3.3).
3. **TODO 3**, `build_prompt`: the text in section 4.
4. **TODO 4**, `INSTRUCTION`: the rules in section 5.
5. **TODO 5**, `answer`: retrieve, build the prompt, ask the model, turn the ids into sources.

Check your work from the repo root:

```
python3 steps/02_rag/check.py
```

- **Part 1** tests `allowed_chunks` and `build_prompt` on four made-up chunks, without Google.
- **Part 2** tests retrieval: for each of the 27 answerable questions, is a right document among the four chunks? Pass mark 90 %, and the expired 2025 policy must never come back.
- **Part 3** asks the full RAG all 30 questions. An answer passes when it has the key fact ("60", "599") and a right source. Pass mark 85 %, and all three no-answer questions must get "I don't know".

The reference solution in this folder passes with 27 of 27 found and 27 of 27 answers right.

7 Experiments, after it passes

- Remove the “I don’t know” rule from your instruction. What does the chat say about renting furniture?
- Set `K = 1`. Which questions break? (Look at the ones of kind `two_docs`.)
- Turn the topic bonus into a hard filter, pick *Returns* and ask about the return pickup in Norway.
- Remove the date check from `allowed_chunks` and ask “Is the return period still 30 days?”.
- Ask in Danish or German. Does the answer follow your language?